西南交通大学
Southwest Jiaotong University

Threads

线程



Xiangyi Chen

xiangyichen@swjtu.edu.cn

Quick Review

回顾

- **Process Definition & Elements**
- **Process states model, eg 2,5,7**
- **Process description**
  - Process image: 进程映像包含
    - ✔ PCB (进程控制块)
    - ✔ User stack (用户栈)
    - ✔ Private user address space (programs, data) (私有用户地址空间) (程序段，数据段)
    - ✔ Shared address space (共享地址空间)
      - PCB: identification, state info and control info
        - PID
        - 状态信息
        - 控制信息
- **Process execution**
  - Modes of execution: kernel and user
    - 执行模式
    - 内核态
    - 用户态
  - Change of process state, 7 steps
    - 进程状态的转换
- **Process in Unix**
  - Preempted 抢占
  - Creation 创建

Chapter 4: Threads

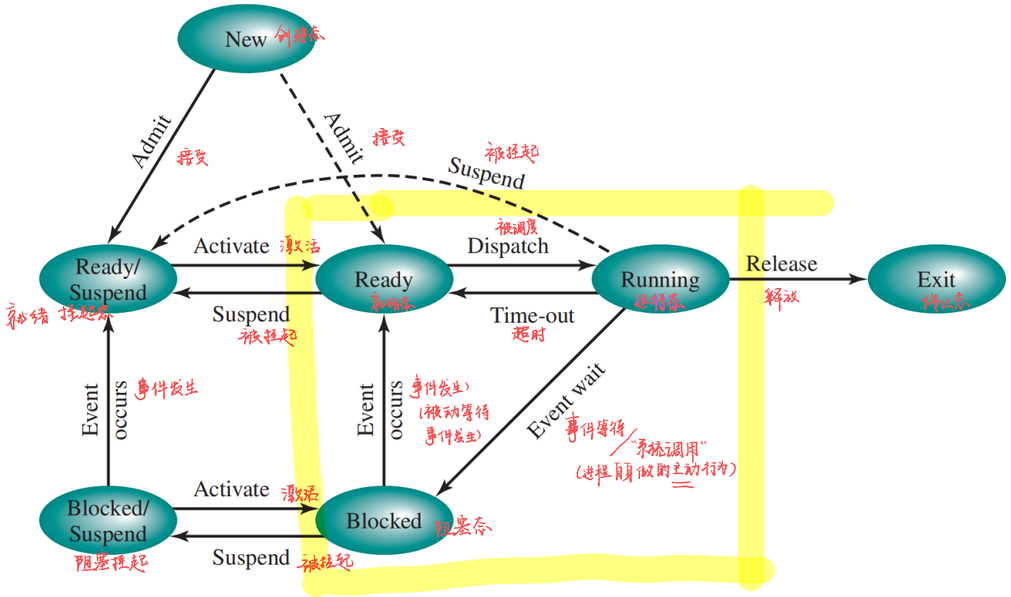
- Process and threads
- Types of Threads
- Multicore / Multithread

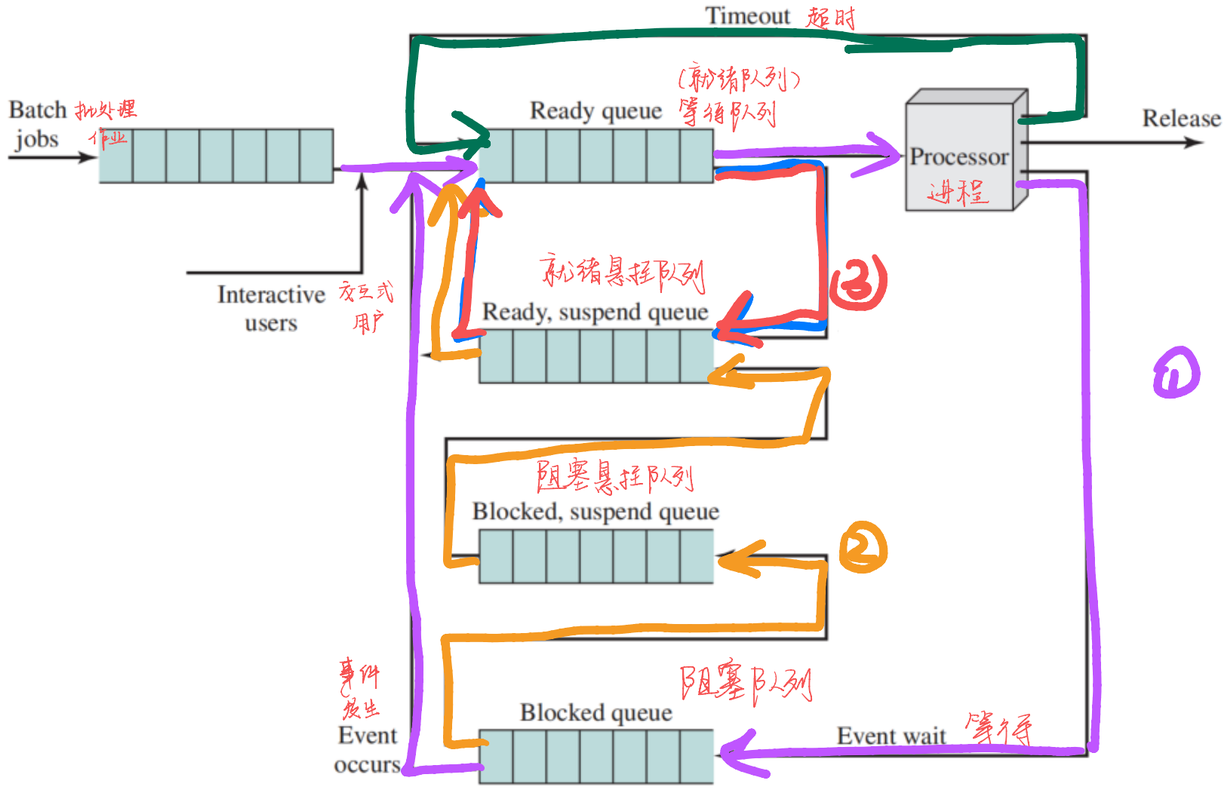
多核/多线程

西南交通大学
Southwest Jiaotong University

Previous Content

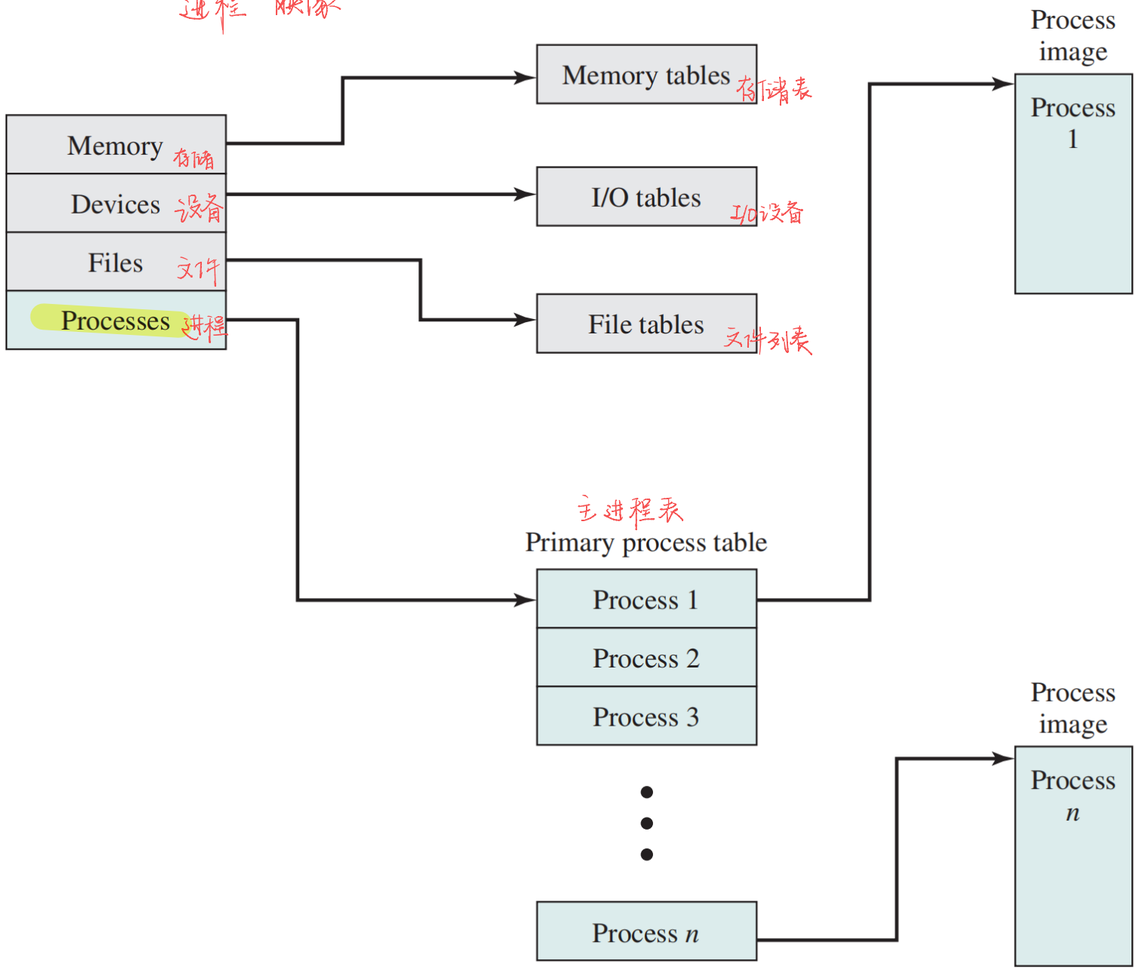
- Process state (2, 5, 7)

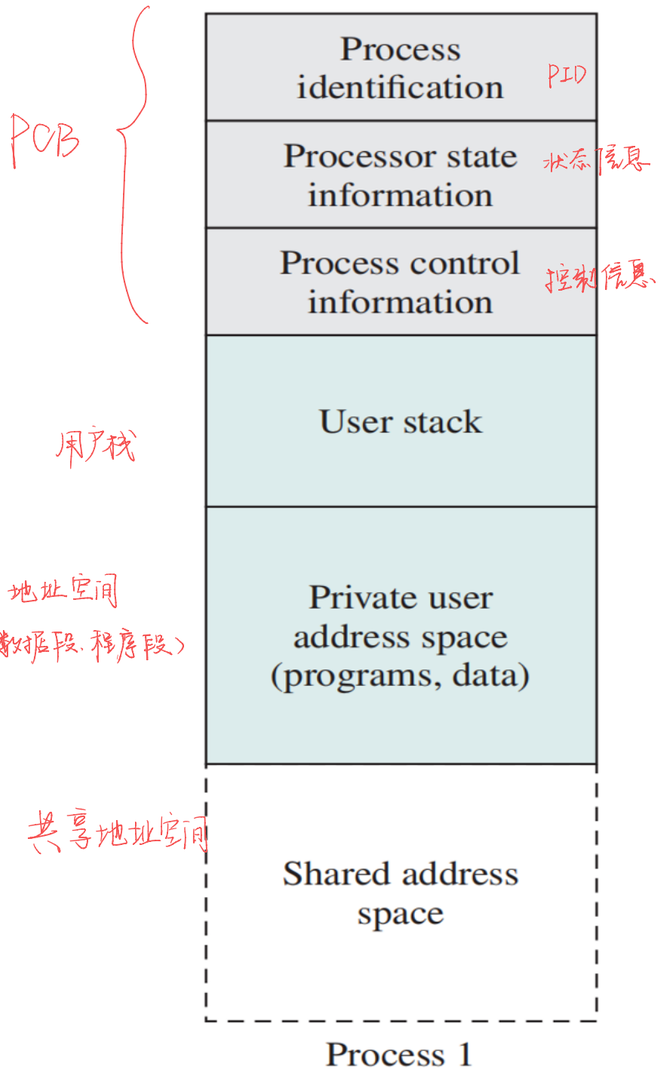




Previous Content

Process Image

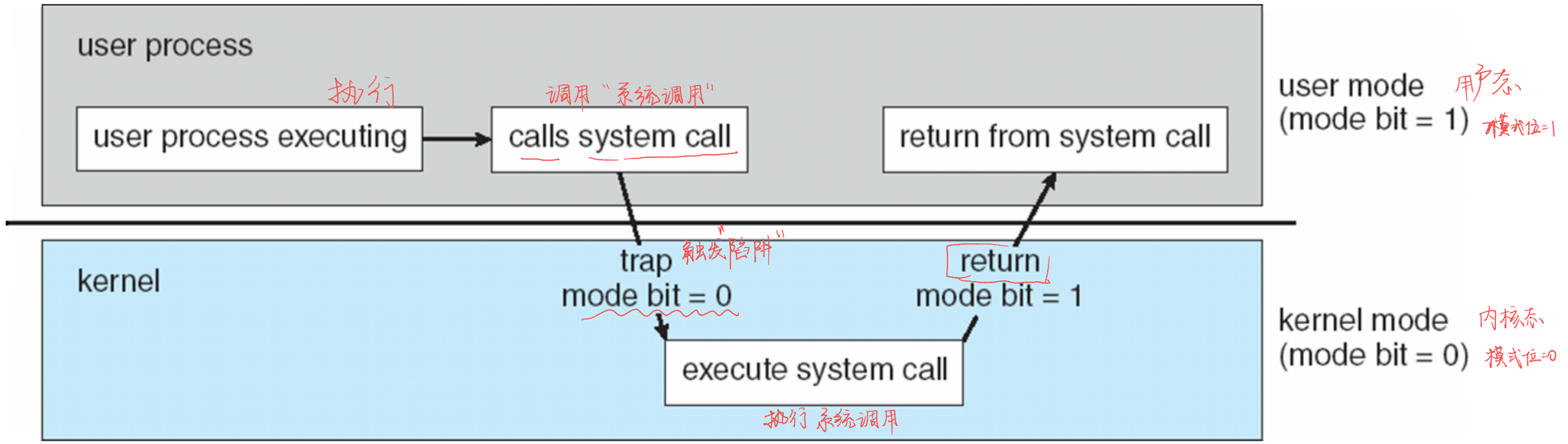




Previous Content

Model of Execution

(执行模式转换图)



Previous Content

西南交通大学 Southwest Jiaotong University

Process Switch 进程切换

1. save the context of the processor 保存上下文
2. update the process control block of the process currently in the running state 更新PCB
3. move the process control block of this process to the appropriate queue 将该PCB移到相应队列
4. select another process for execution 选择另外一个进程准备执行
5. update the process control block of the process selected 更新被选中的PCB
6. update memory management data structures 更新内存管理数据结构
7. restore the context of the processor to that which existed at the time the selected process was last switched out 恢复上下文 恢复至被选中进程上次被切换出的状态

A Mp3 application


```
main( )
{
      while(TRUE )
      {
           Read( );
           Decompress( );
           Play( );
       }
}
Read( ) { ... }
Decompress( ) { ... }
Play( ) { ... }
```

A Mp3 application


```
程序1
main( )
{
   while(TRUE )
   {
       Read( );
    }
}
Read( ) { … }
```

```
程序2
main( )
{
   while(TRUE )
   {
      Decompress( );
    }
}
Decompress( ) { … }
```

```
程序3
main( )
{
   while(TRUE )
   {
      Play( );
   }
}
Play( ) { … }
```

4.1 Processes and Threads

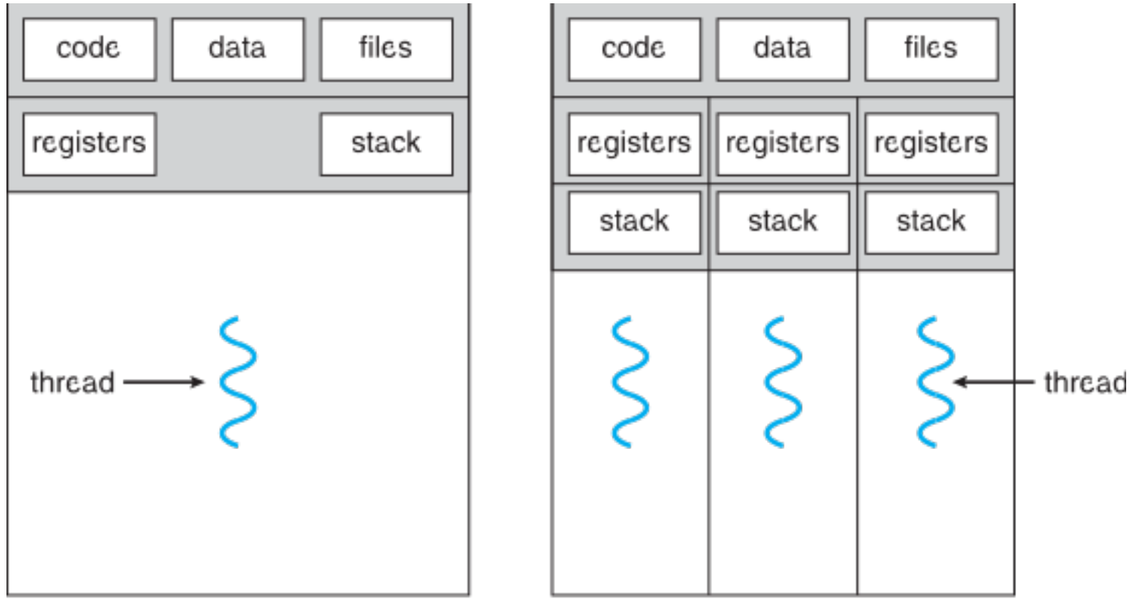
西南交通大学 Southwest Jiaotong University

Processes have two characteristics:

- **Resource Ownership** 资源拥有权
  - Process includes a virtual address space to hold the process image 虚拟地址空间 手抓
  - OS performs a protection function to prevent unwanted interference between processes with respect to resources, such as processors, main memory, files and I/O resources 保护功能
- **Scheduling/Execution** 调度/执行
  - Follows an execution path that may be interleaved with other processes 执行路径可交错进行(时间片)
  - A process has an execution state (Running, Ready, etc.) and a dispatching priority 调度优先级 dispatching
  - A process is scheduled and dispatched by the OS 被OS调度和分配

Processes and Threads

- The unit of dispatching is referred to as a thread or lightweight process
- The unit of resource ownership is referred to as a process or task
- Multithreading - The ability of an OS to support multiple, concurrent paths of execution within a single process

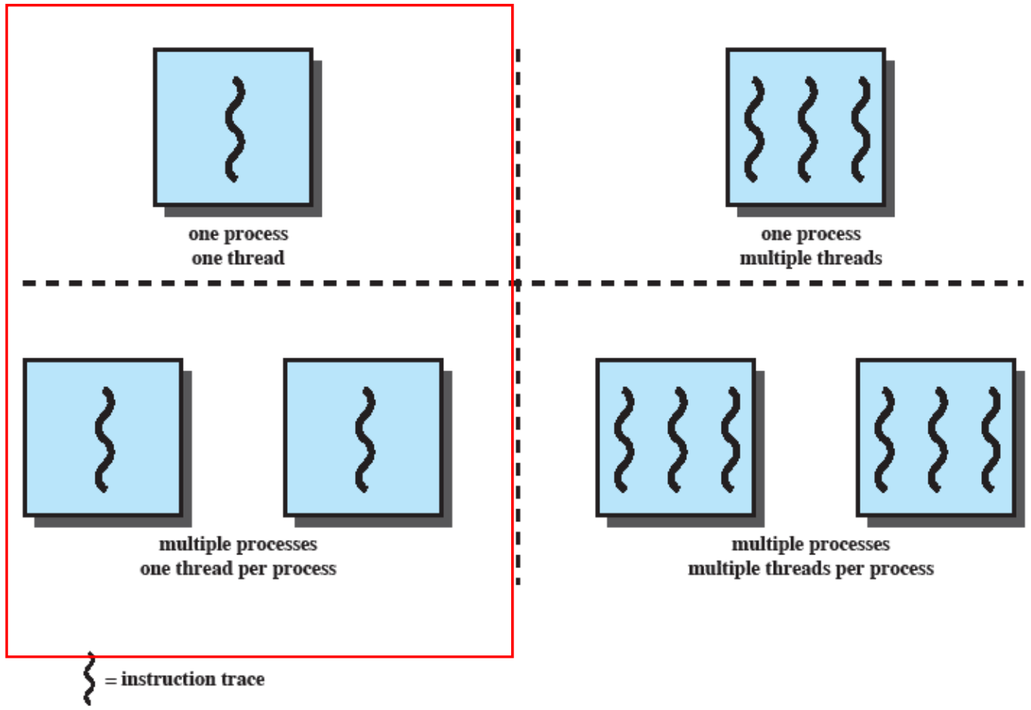


single-threaded process　　multithreaded process

Single Threaded Approaches

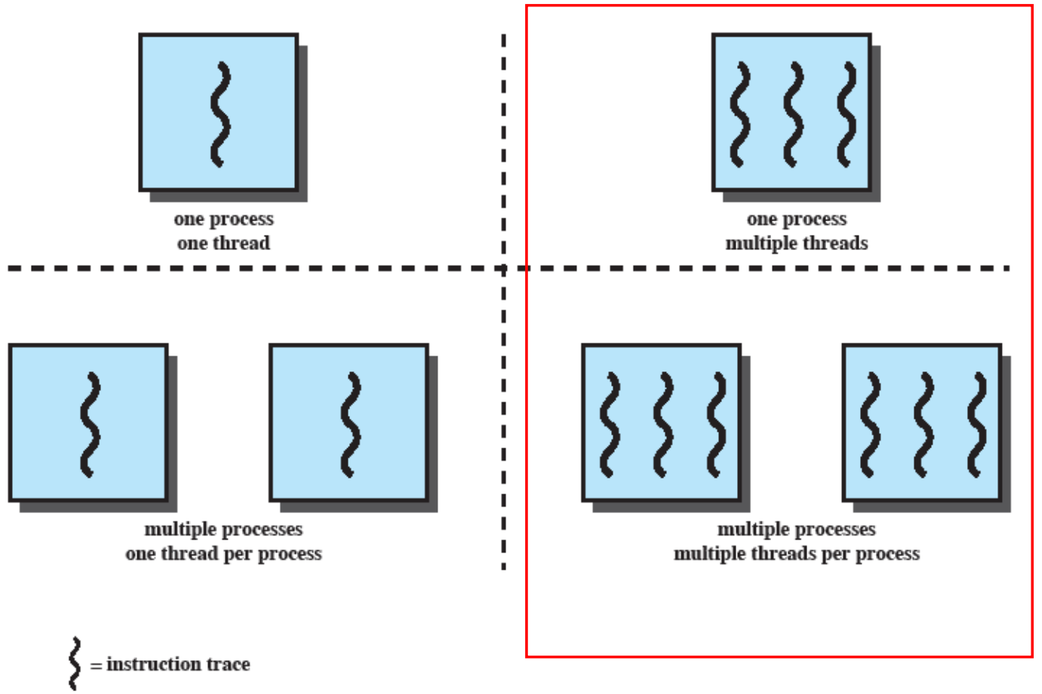
- **A single thread of execution per process**
  - the concept of a thread is not recognized

- **MS-DOS**
- **Traditional Unix**



Multithreaded Approaches

- The right half depicts multithreaded approaches
- A Java run-time environment is an example of a system of one process with multiple threads
- Windows, Solaris and modern Unix are multiple processes with multiple threads



One or More Threads in a Process

西南交通大学 Southwest Jiaotong University

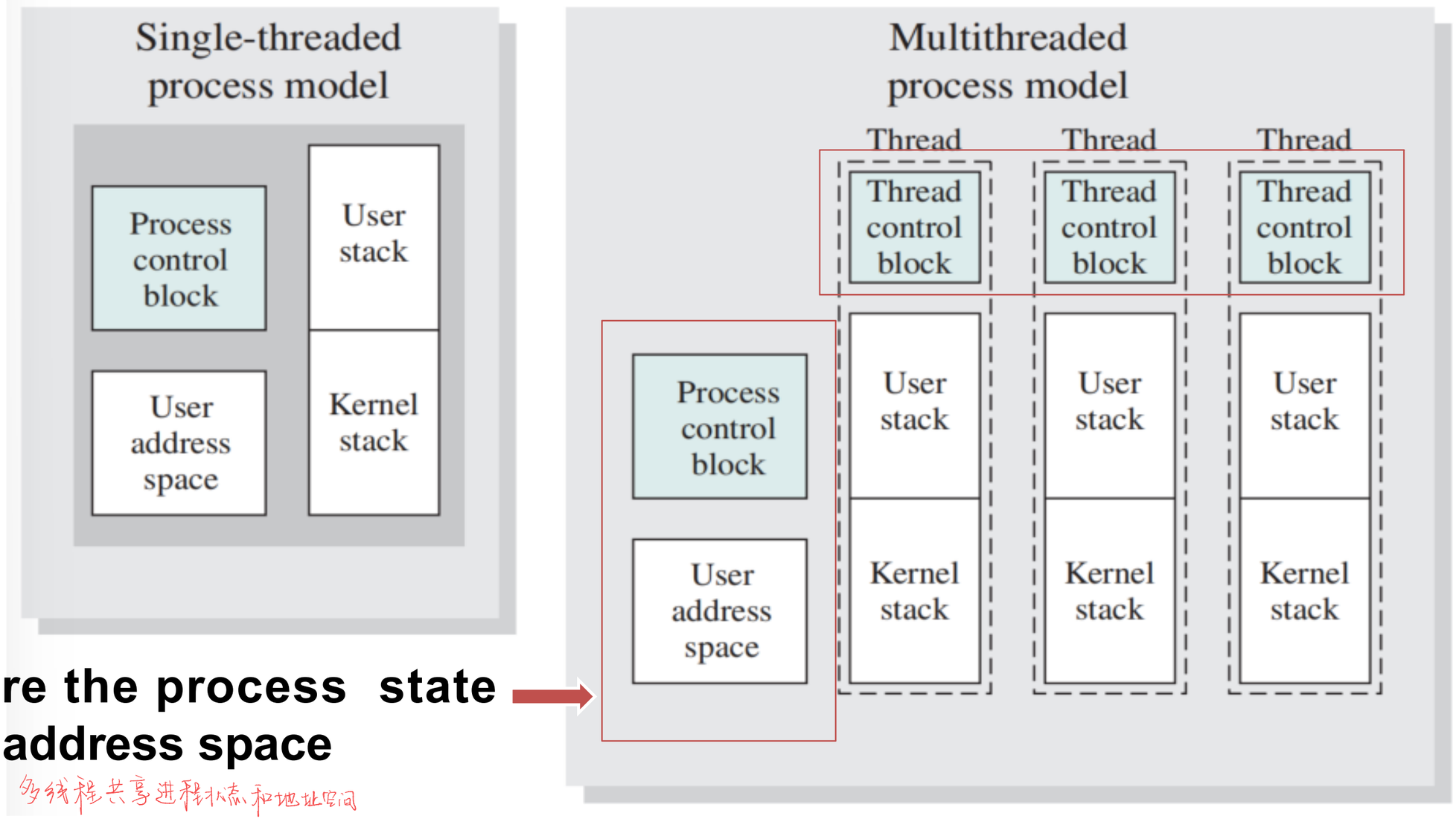
Each process has

- A virtual address space that holds the process imag
- The unit or resource allocation and a unit of protection
- Protected access to:
  - processors
  - other processes
  - files
  - I/O resources

Each thread has:

- an execution state (Running, Ready, etc.)
- saved thread context when not running
- an execution stack
- some per-thread static storage for local variables
- access to **the memory and resources of its process** (all threads of a process share this)

Single-Threaded and Multithreaded Process Models



- Share the process state and address space

多线程共享进程状态和地址空间

Benefits of Threads

西南交通大学
Southwest Jiaotong University

- Less time to create a new thread than a process
- Less time to terminate a thread than a process
- Switching between two threads takes less time than switching between processes
- Threads enhance efficiency in communication between programs by avoiding kernel intervention

A collection of thread is more efficient to a collection of separate processes!

Thread Usage

西南交通大学 Southwest Jiaotong University

Multithread in Uuiprocessor system

- To simplify the structure of a program that is logically doing several different functions.
  - Foreground and background work
  - Asynchronous processing
  - Speed of execution
  - Modular program structure

Multithread in Multiprocessor system

- Multiple threads in the same process can be executing simultaneously on different processors.

Thread States

- **The key states for a thread are:**
  - Running
  - Ready
  - Blocked

suspend states are process-level concepts.

- **Thread operations associated with a change in thread state are:**
  - Spawn: a thread within a process may spawn another thread within the same process. The new thread is placed on the **Ready queue**.
  - Block: waiting for an event, **running->block**
  - Unblock: waited event occurs , **block->ready queue**
  - Finish: a thread completes, the register context and stacks are **deallocated**.

Why not suspend state? How about swapping a thread ?

Thread Execution

西南交通大学 Southwest Jiaotong University

- **In an OS that supports threads:**
  - scheduling and dispatching is done on a **thread basis**
  - Most of the state information dealing with execution are maintained in **thread-level data structures**
- **Some actions affect all of the threads in a process**:
  - **suspending** a process involves suspending all threads of the process
  - **termination** of a process terminates all threads within the process
- **It is necessary to synchronize(同步) the activities of the various threads**
  - all threads of a process share the same address space and other resources
  - any alteration of a resource by one thread affects the other threads in the same process

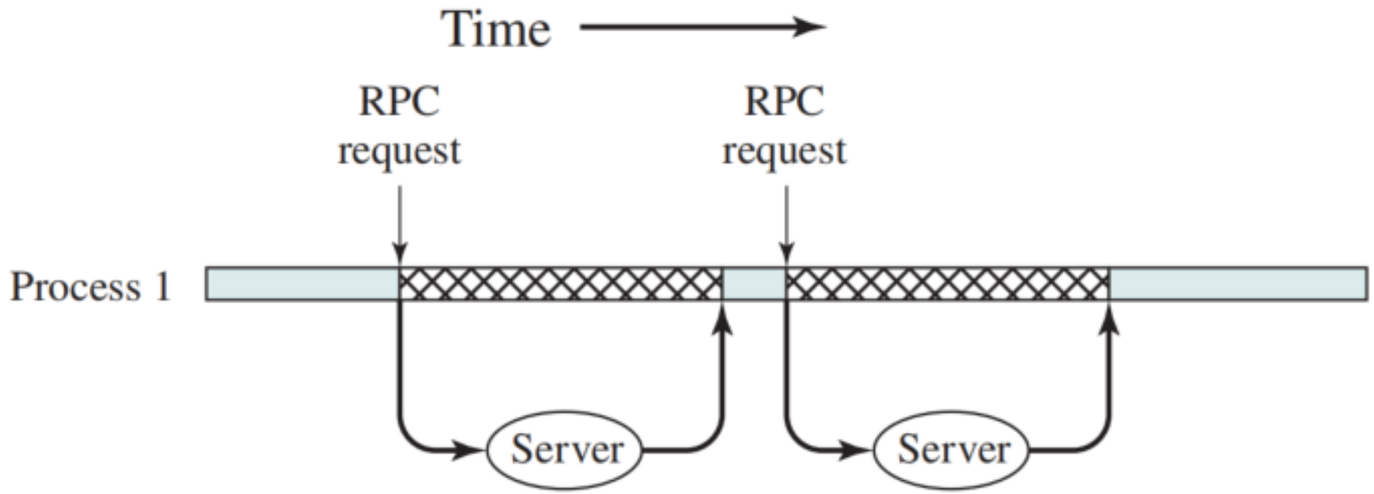
修改

Whether the blocking of a thread results in the blocking of the entire process?

Yes

RPC Using Single Thread

- First, considering the performance benefits of threads that do not block an entire process
- In a single-threaded program, the results are obtained in sequence, so the program has to wait for a response from each server in turn

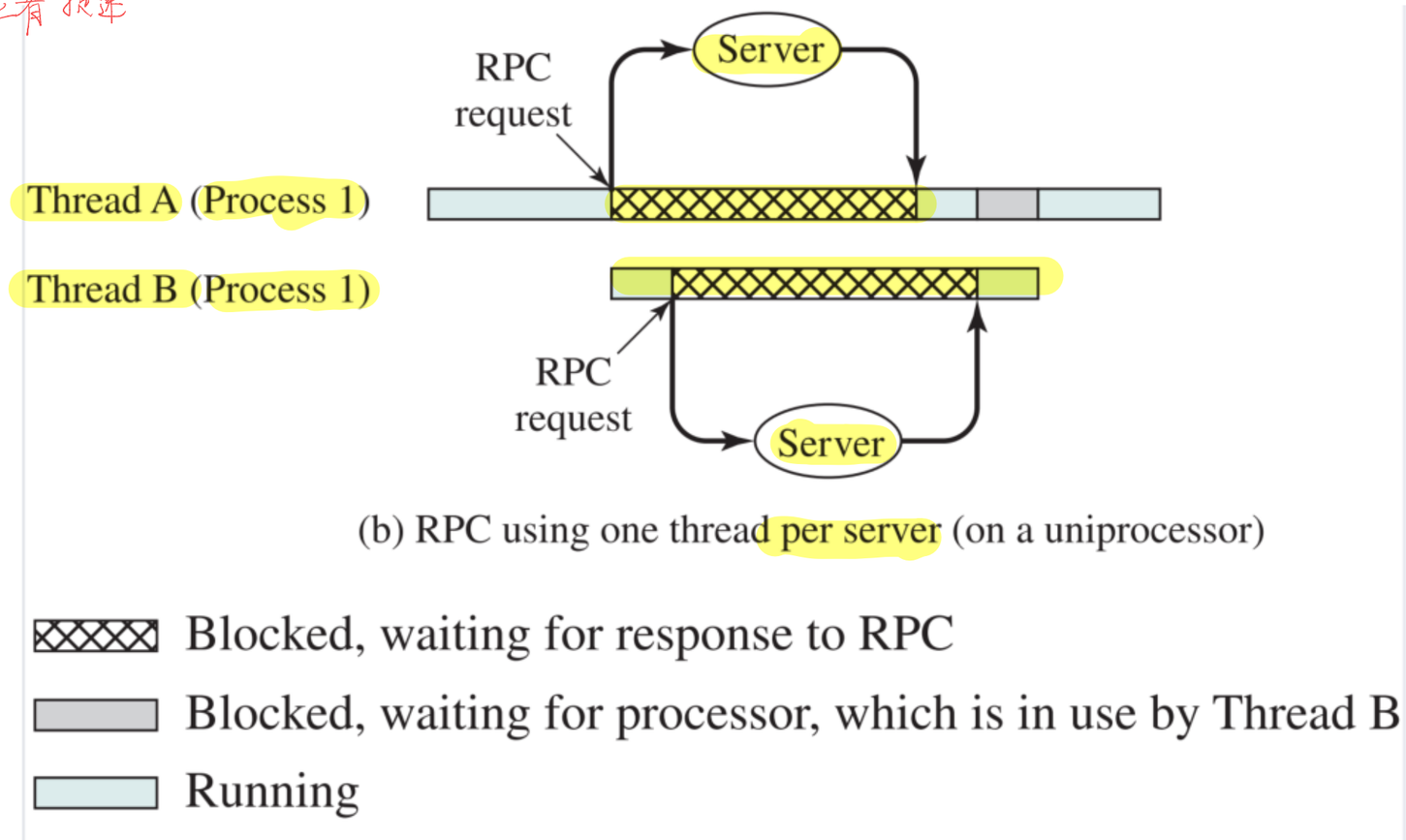


A program that performs two remote procedure calls (RPCs)2 to two different hosts to obtain a combined result.

不同的主机

RPC Using One Thread per Server

- Rewriting the program to use a separate thread for each RPC results in a substantial speedup

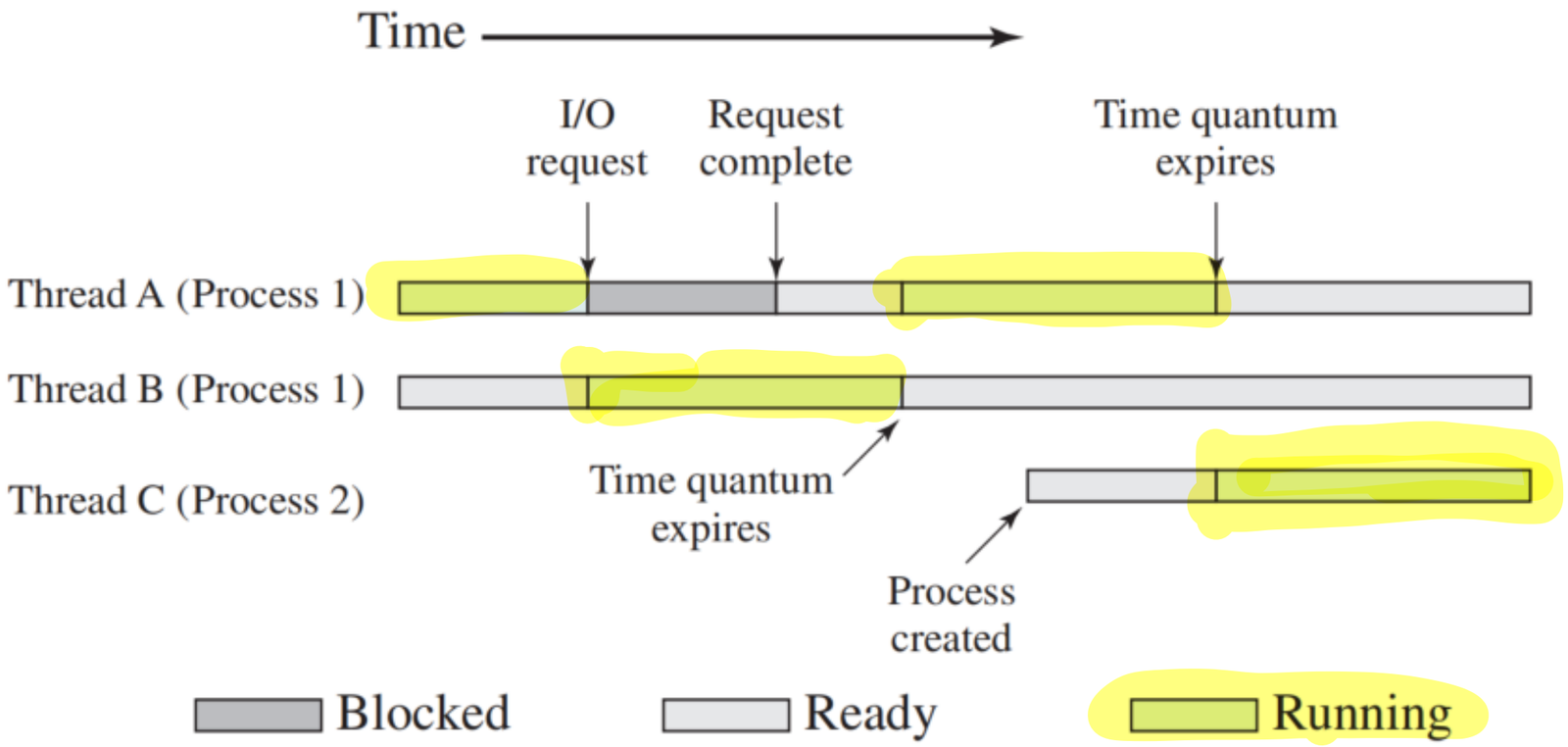


(b) RPC using one thread per server (on a uniprocessor)

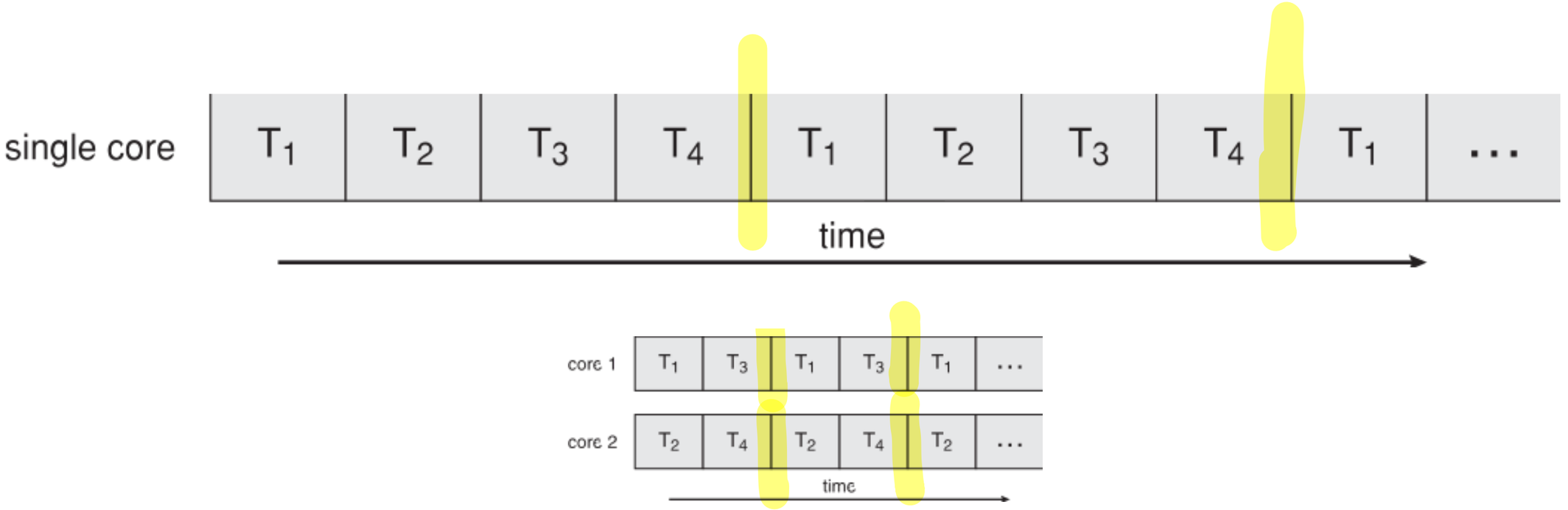
Multithreading on a Uniprocessor

单处理器上的多线程

- On a uniprocessor, multiprogramming enables the interleaving of multiple threads within multiple processes



Concurrency（并发）v.s. Parallelism（并行）



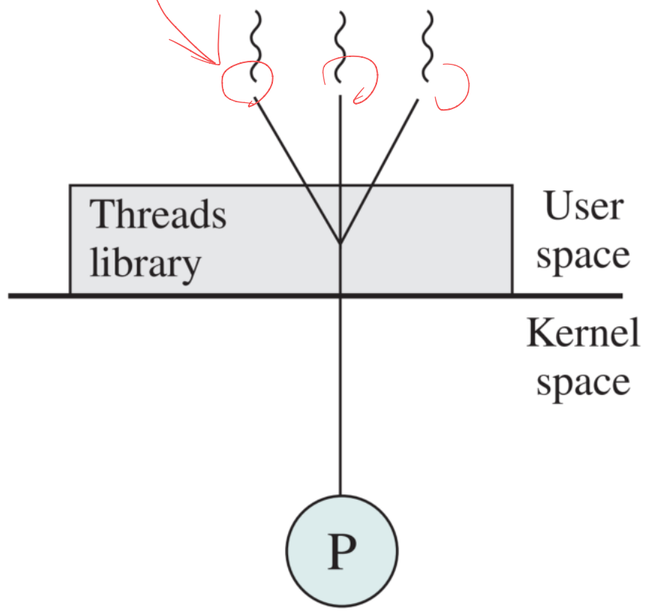
4.2 Types of Threads

- User Level Thread (ULT)
- Kernel level Thread (KLT)

User mode v.s kernel mode?

User-Level Threads (ULTs)

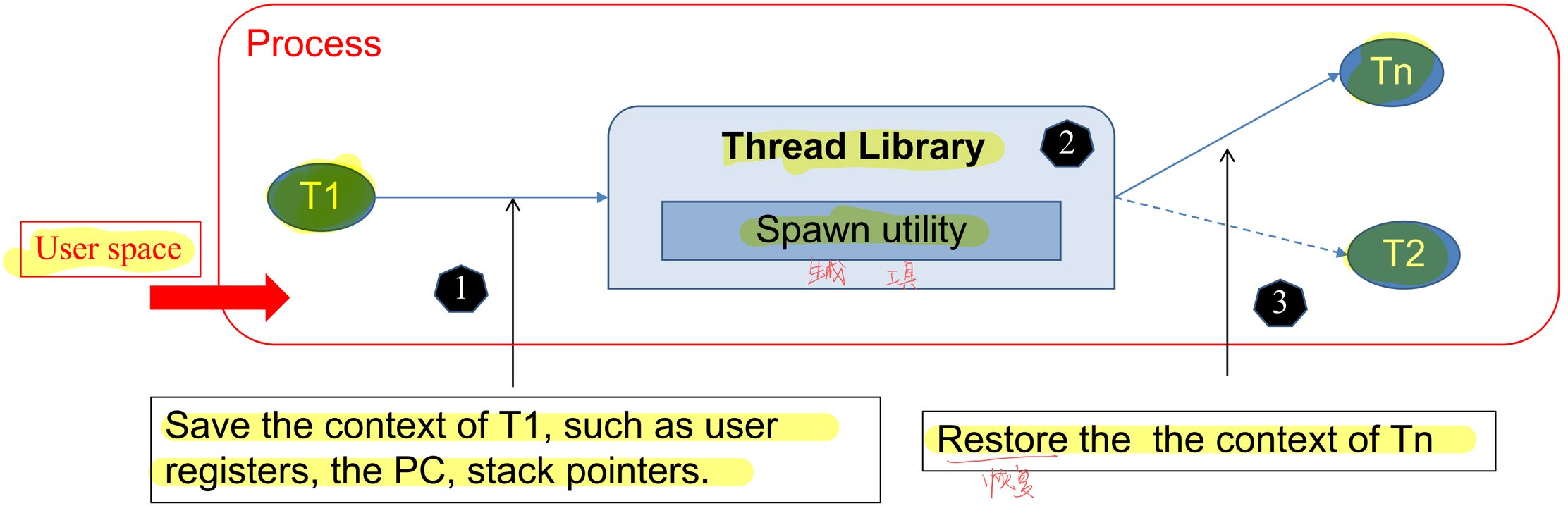
- All thread management is done by the application
- The kernel is not aware of the existence of threads
- Thread library provides programmer with API for creating and managing threads



Pure user–level

Thread Management of ULTs

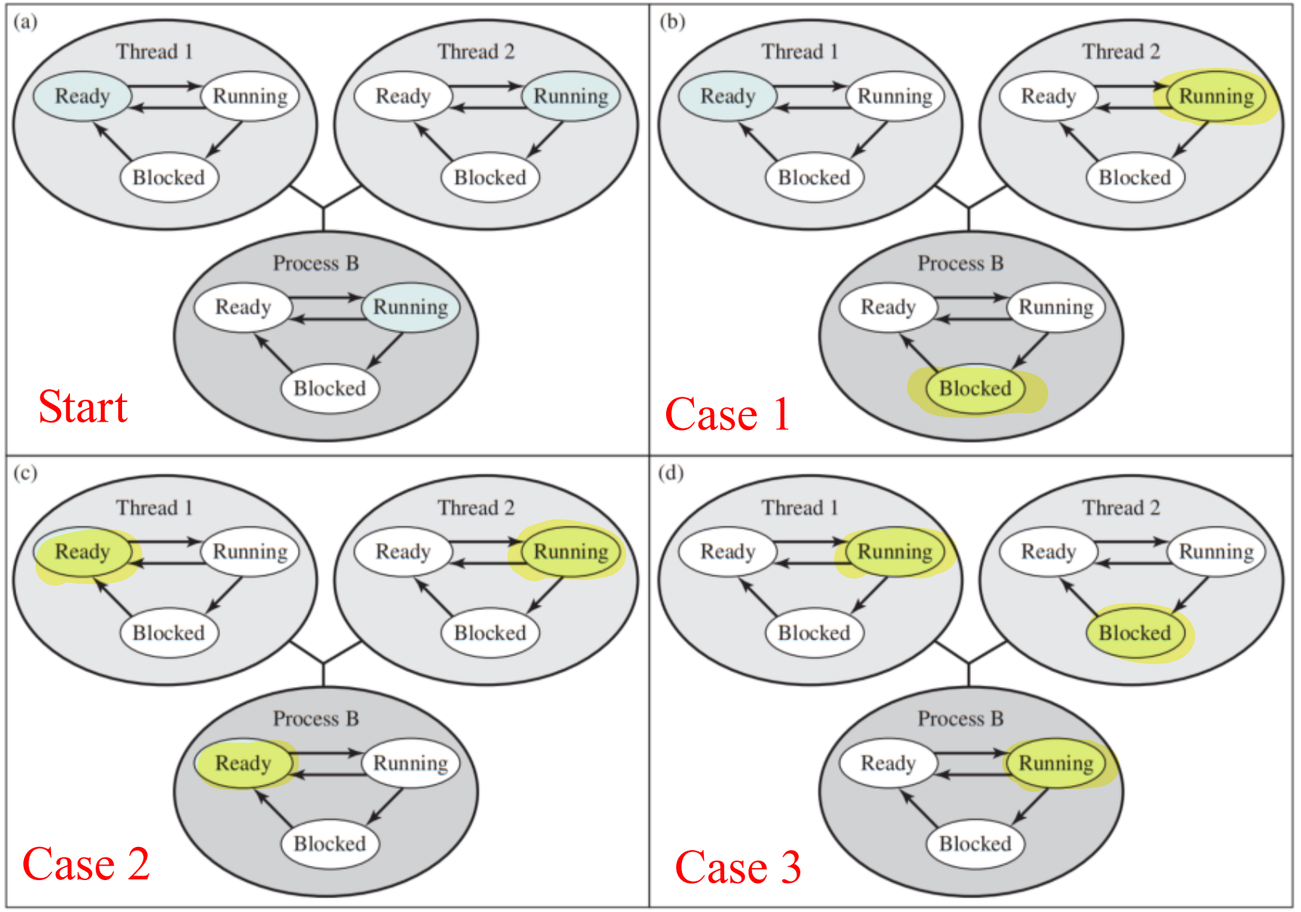
- Procedures of thread switching



Relationships Between ULT States and Process States

西南交通大学 Southwest Jiaotong University

- **Case1: T2 makes a system call, such request I/O**
  - For processor: thread 2 is executed
  - For threads library: , according to the data structure maintained by the threads library, thread 2 is still in the Running state by the threads library.
- **Case 2: A clock interrupt**
- **Case 3: thread 2 waits the results of thread 1;**



Examples of the Relationships between User-Level Thread States and Process States

Advantages of ULTs

西南交通大学 Southwest Jiaotong University

- **Thread switching does not require kernel mode privileges**
  - all of the thread management data structures are within the user address space of a single process.
  - saves the overhead of two mode switches (user to kernel; kernel back to user).

调度 被应用程序定制

- **Scheduling can be application specific**
  - The scheduling algorithm can be tailored to the application without disturbing the underlying OS scheduler.

底层操作系统调度器

- **ULTs can run on any OS**
  - No changes are required to the underlying kernel to support ULTs. The threads library is a set of application-level functions shared by all applications.

Disadvantages of ULTs

- when a ULT executes a system call, not only is that thread blocked, but all of the threads within the process are blocked (Case 1)

阻塞

- In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing

Solution

- Writing an application as multiple processes rather than multiple threads.
  - Introduce the overhead of process switching

会产生进程切换的开销。

- Jacketing(封装)

转换

  - converts a blocking system call into a non-blocking system call (using the thread library to passes control to another thread rather than the OS scheduler)

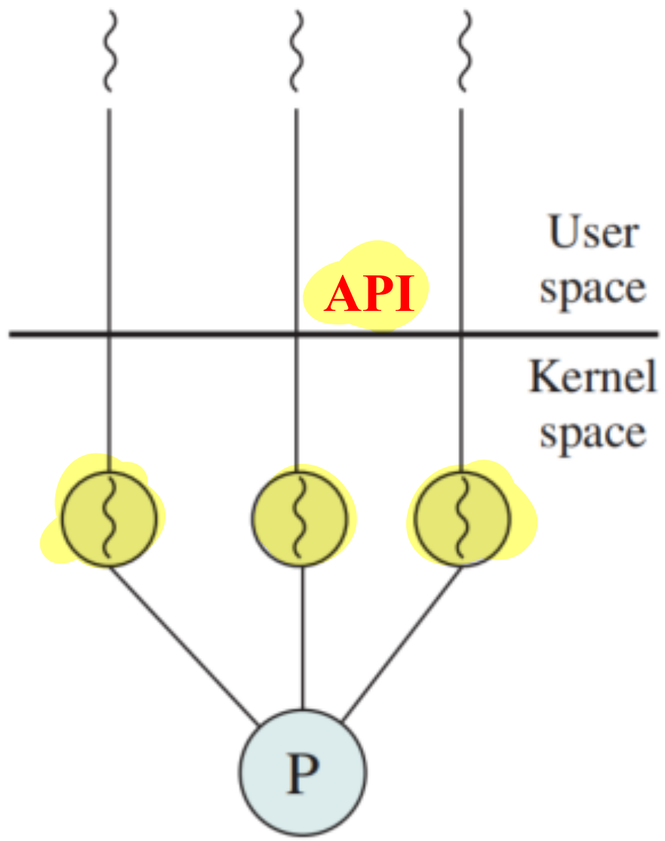
通过线程库将控制权传递给另一个线程

Kernel-Level Threads (KLTs)

西南交通大学 Southwest Jiaotong University

- **Thread management is done by the kernel**
  - no thread management is done by the application
  - The kernel maintains context information for the process as a whole and for individual threads within the process.
  - Scheduling by the kernel is done on a thread basis.

- **Examples – virtually all general purpose OS**
  - Windows
  - Solaris
  - Linux
  - Tru64 UNIX
  - Mac OS X



(b) Pure kernel–level

Advantages of KLTs

西南交通大学
Southwest Jiaotong University

内核、可以将同一个进程的线程调度到多个处理器上

- The kernel can simultaneously schedule multiple threads from the same process on multiple processors
- If one thread in a process is blocked, the kernel can schedule another thread of the same process
- Kernel routines (内核例程) can be multithreaded

Disadvantage of KLTs

- The transfer of control from one thread to another within the same process requires a mode switch to the kernel

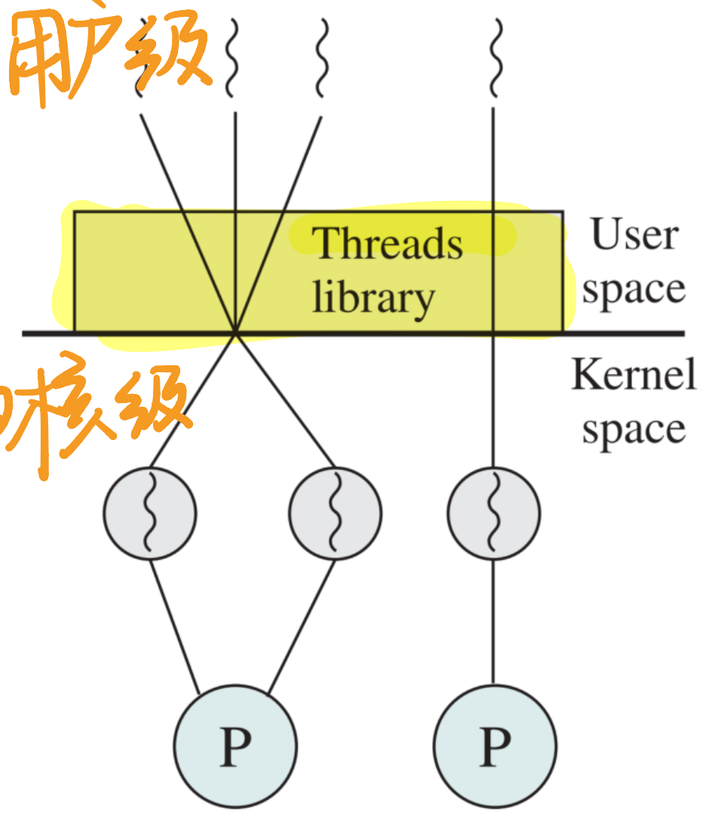
Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840

Thread and Process Operation Latencies (*us*)

- **Full Fork**：the time to create, schedule, execute, and complete a process/thread that invokes the null procedure (i.e., the overhead of forking a process/thread)
- **Signal-Wait**：the time for a process/thread to signal a waiting process/thread and then wait on a condition (i.e., the overhead of synchronizing two processes/threads together).

Combined Approaches

- Thread creation is done in the user space
- Bulk of scheduling and synchronization of threads is by the within an application
- The multiple ULTs from a single application are mapped onto some (smaller or equal) number of KLTs.
- The programmer may adjust the number of KLTs for a particular application and processor to achieve the best overall results.



(c) Combined

Relationship Between Threads and Processes

西南交通大学 Southwest Jiaotong University

Threads: Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	It combines attributes of M:1 and 1:M cases.	TRIX

内核级

用户级

组合

4.3 Multicore / Multithread

- The potential performance benefits of a multicore organization depend on the ability to effectively exploit the parallel resources available to the application.
- A single application running on a multicore system

Amdahl's Law (阿姆达尔定律)

$$\text{Speedup} = \frac{\text{time to execute program on a single processor}}{\text{time to execute program on } N \text{ parallel processors}} = \frac{1}{(1-f) + \frac{f}{N}}$$

1-f: a fraction of the serial execution time

f: a fraction of the parallelizable execution time with no scheduling overhead

Performance Effect of Multiple Cores

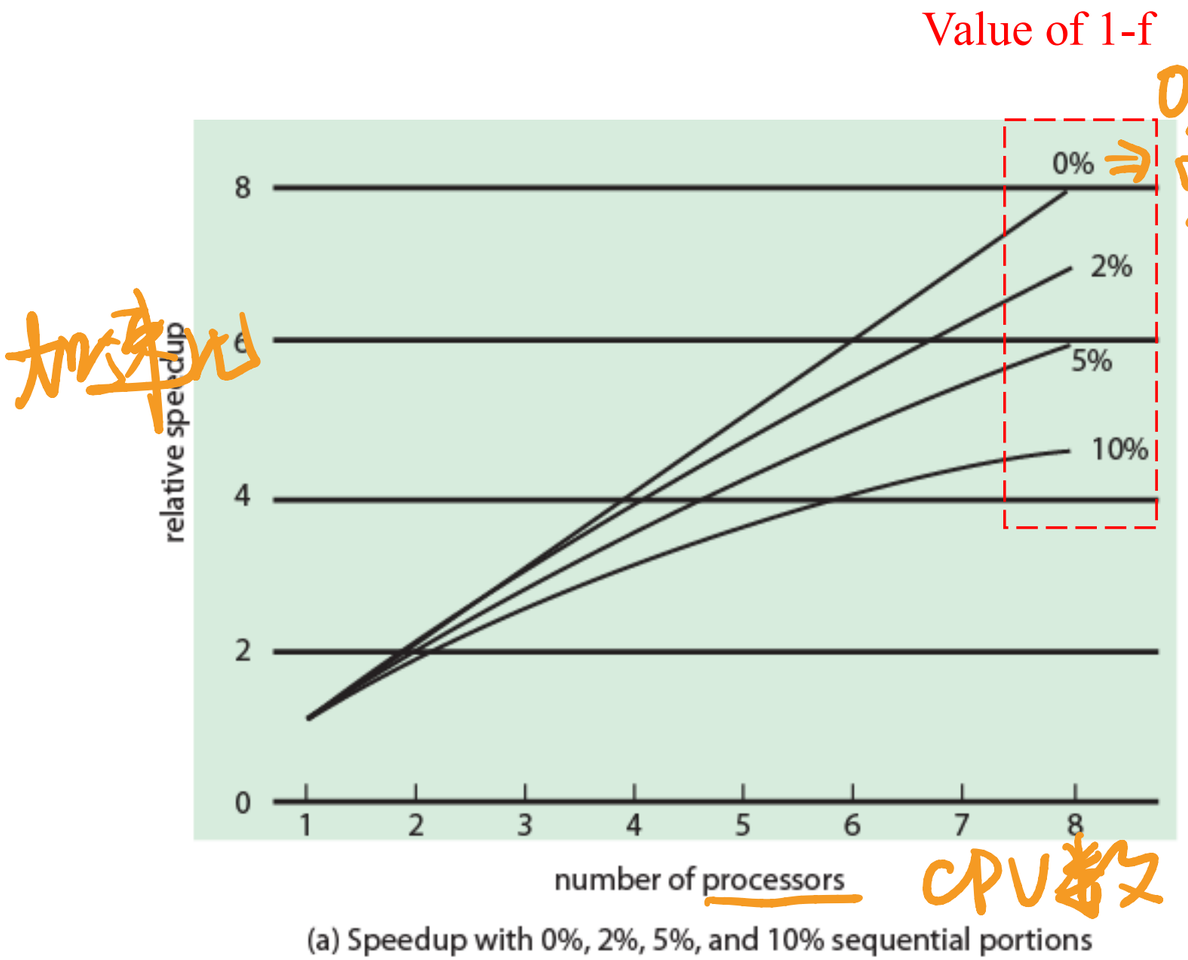


Figure 4.7 (a)

e.g., communication and cache overhead

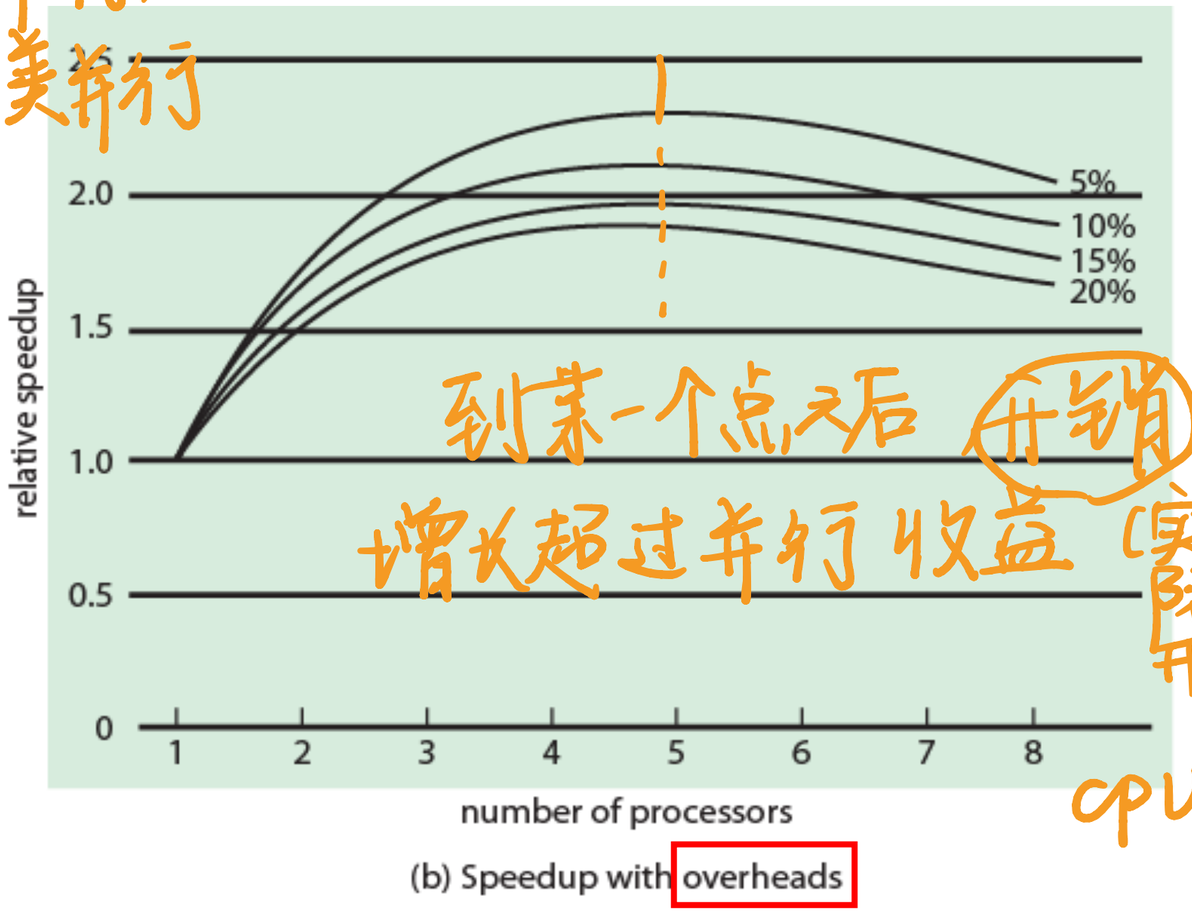
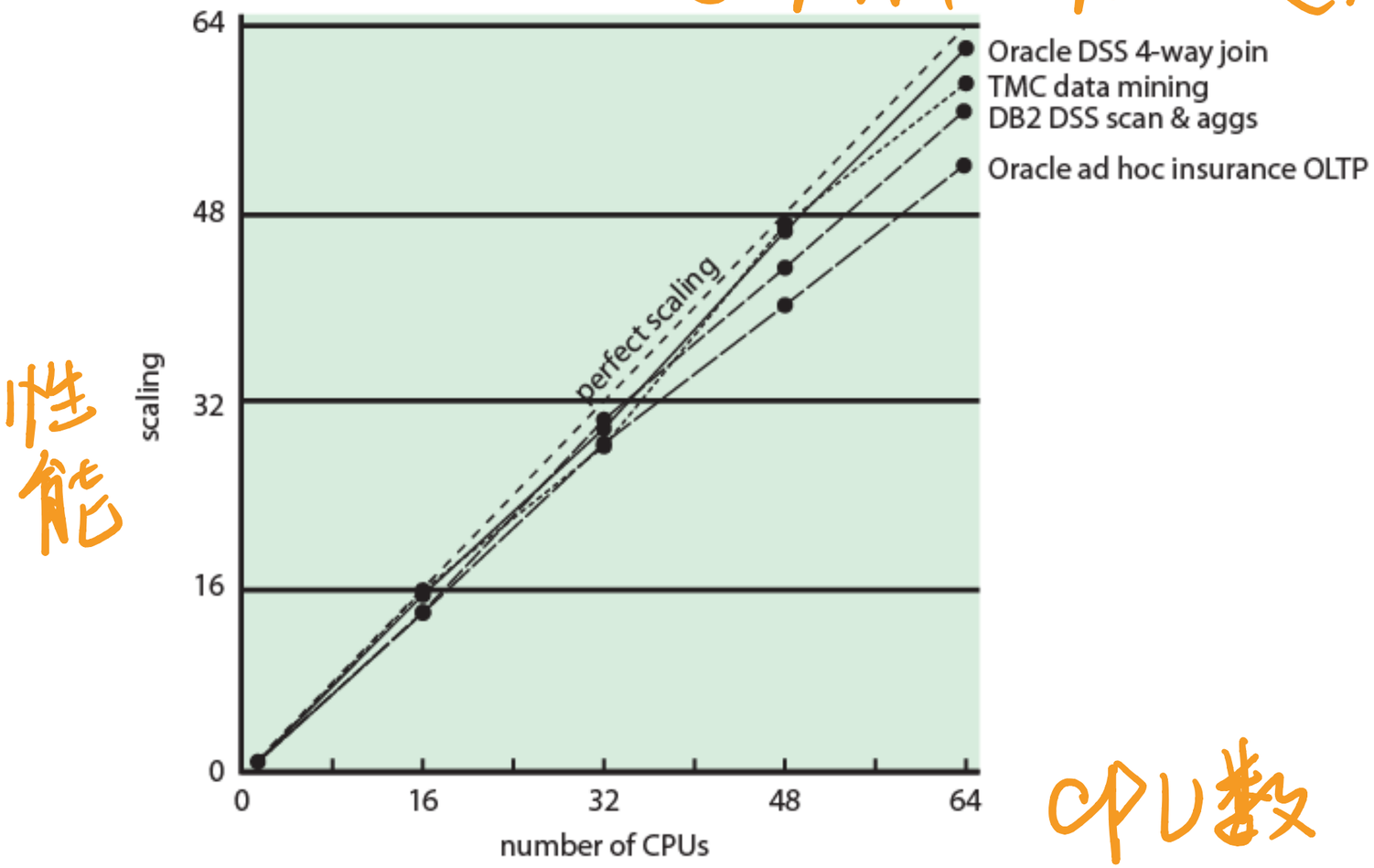


Figure 4.7 (b)

Example of Multicore for Database

数据库中多核的运用



性能

CPU数

Figure 4.8 Scaling of Database Workloads on Multiple Processor Hardware

Benefited Applications from Multicore

西南交通大学 Southwest Jiaotong University

- **Multithreaded native applications**
  - characterized by having a small number of highly threaded processes
- **Multiprocess applications**
  - characterized by the presence of many single-threaded processes
- **Java applications**
- **Multi-instance applications**
  - multiple instances of the application in parallel

Linux Process/Thread Model

- Running
- Ready
- Interruptible(可中断)
- Uninterruptible
- Stopped(停止)
- Zombie (僵死)

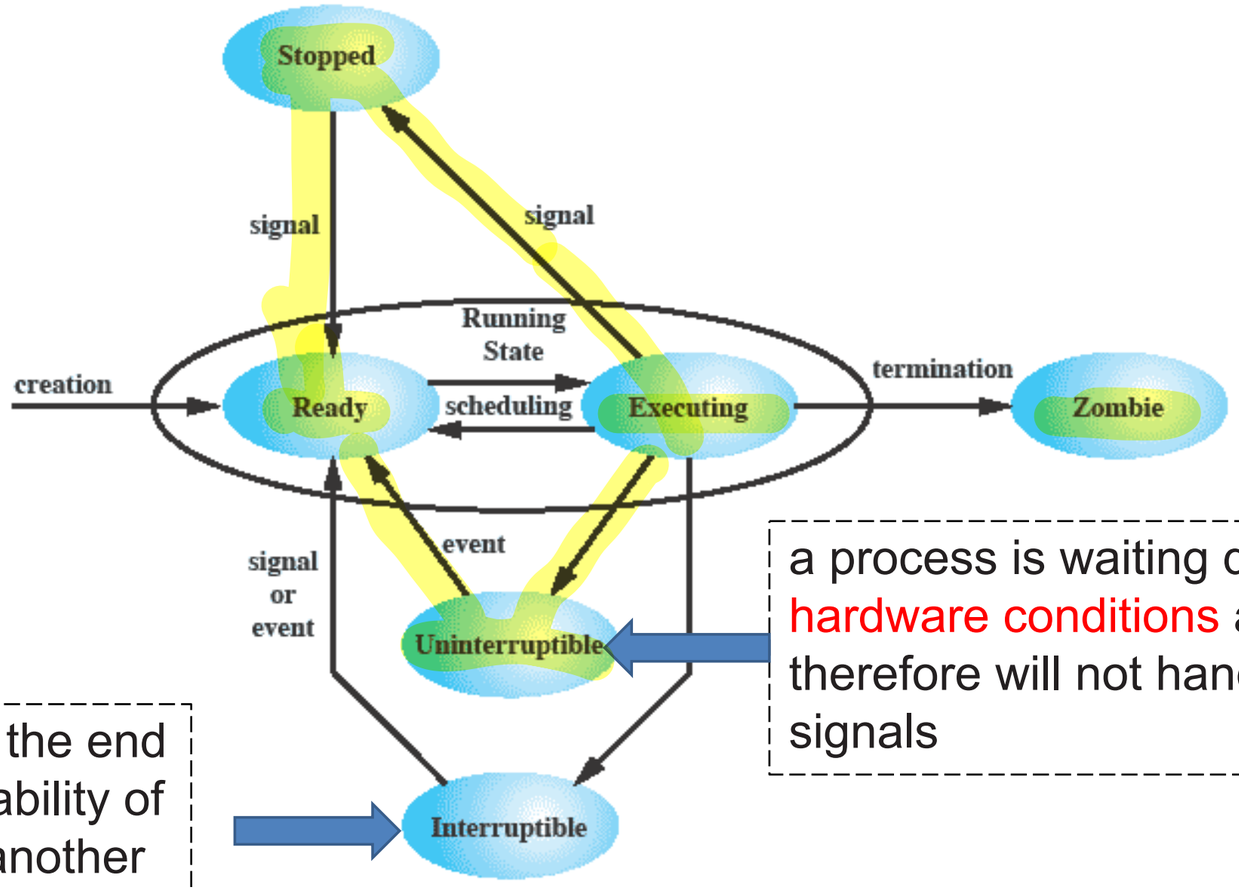


Figure 4.18 Linux Process/Thread Model

waiting for an event, such as the end of an I/O operation, the availability of a resource, or a signal from another process.

a process is waiting directly on hardware conditions and therefore will not handle any signals

Linux Process/Thread Management

- **Traditional UNIX systems support a single thread of execution per process**
  - Instead, applications are written with a set of user-level library functions (e.g., pthread libraries), with all of the threads mapping into a single kernel-level process
- **Modern UNIX systems typically provide support for multiple kernel-level threads per process (clone())**
  - does not recognize a distinction between threads and processes
    - User-level threads (lightweight processes) are mapped into kernel-level processes
  - Multiple user-level threads that constitute a single user-level process are mapped into Linux kernel-level processes that share the same group ID.
    - share resources such as files and memory, and to avoid the need for a context switch when the scheduler switches among processes in the same group. **(clone() create a process, P208)**
    - Individual user stack

Summary

- **User-level threads**
  - created and managed by a threads library that runs in the user space of a process
  - a mode switch is not required to switch from one thread to another
  - only a single user-level thread within a process can execute at a time
  - if one thread blocks by service call, the entire process is blocked
- **Kernel-level threads**
  - threads within a process that are maintained by the kernel
  - a mode switch is required to switch from one thread to another
  - multiple threads within the same process can execute in parallel on a multiprocessor
  - blocking of a thread does not block the entire process
- **Process/related to resource ownership**
- **Thread/related to program execution**